

Object-Oriented Programming in Java

Emily Lucia Antosch

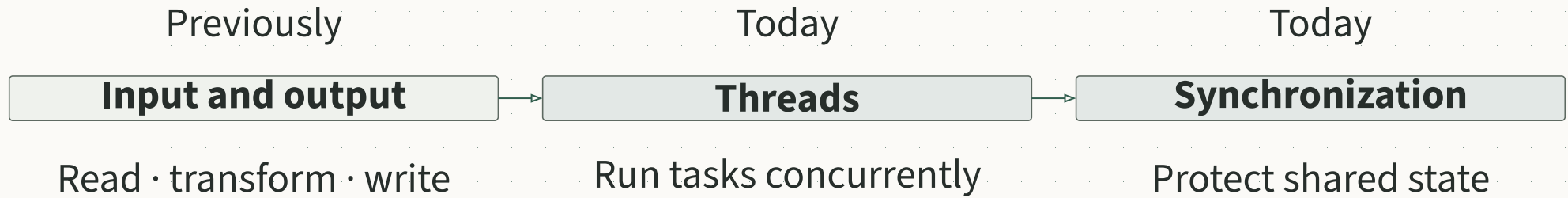
18.09.2026

Today's route

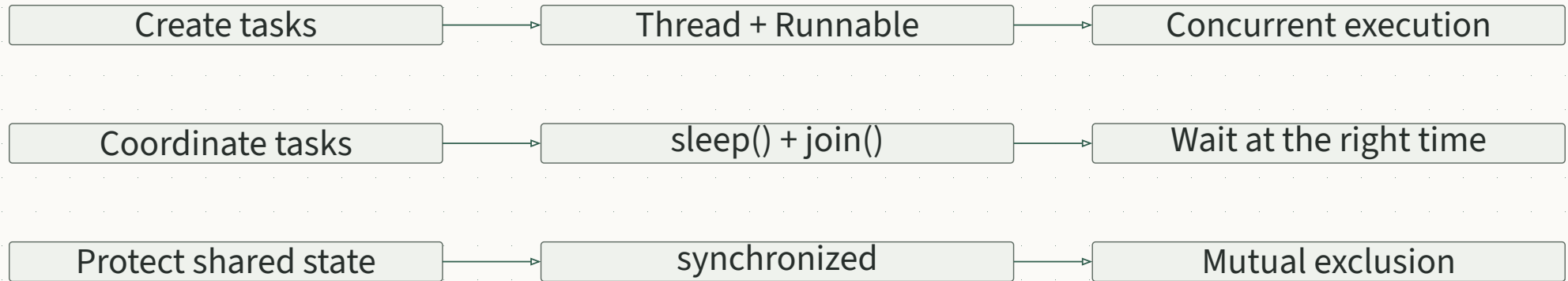
1. Introduction	2
2. Parallel Processing	5
3. Class-Based Threads	13
4. Interface-Based Threads	19
5. Thread States and Waiting	25
6. Synchronization	41
7. License Notice	49

1. Introduction

From I/O to Concurrent Tasks



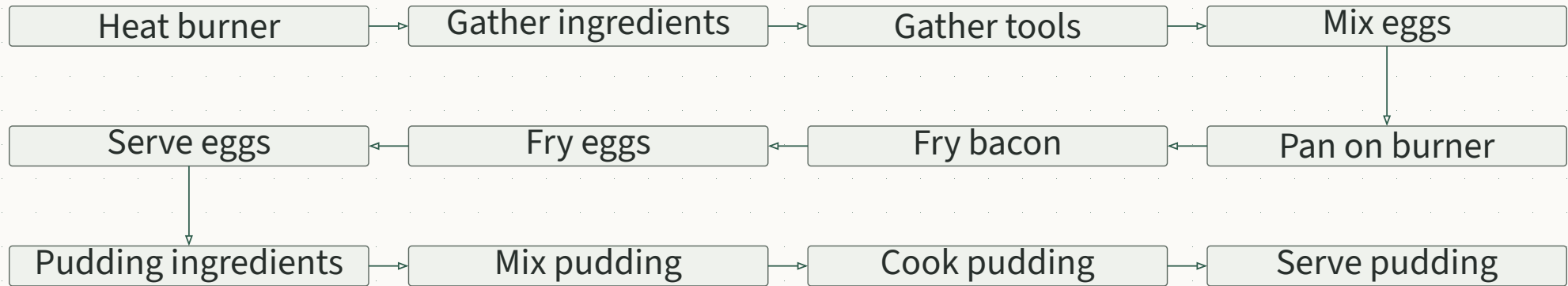
Create, Coordinate, Protect



Examples use platform threads. Swing examples omit imports from `javax.swing` and `java.awt`.

2. Parallel Processing

Scrambled Eggs and Pudding: One Cook

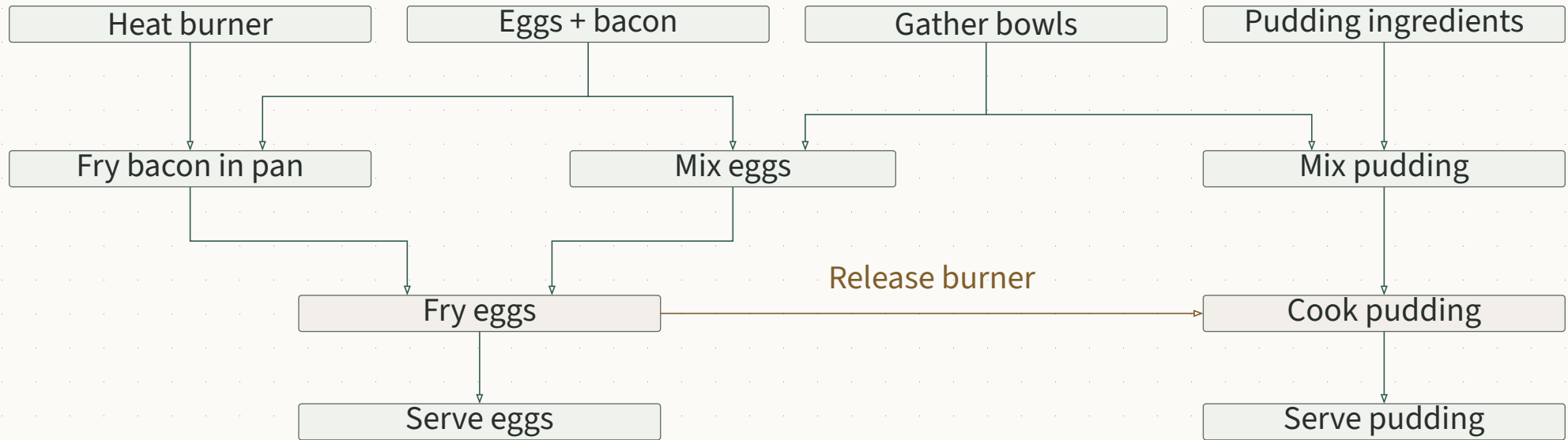


One cook • one task at a time



How would four cooks share the work with only one burner?

Four Cooks, One Burner



Prepare in parallel; use the burner in sequence. Arrows show prerequisites, not durations.

What Limits Parallel Work?



Dependency: wait for a result



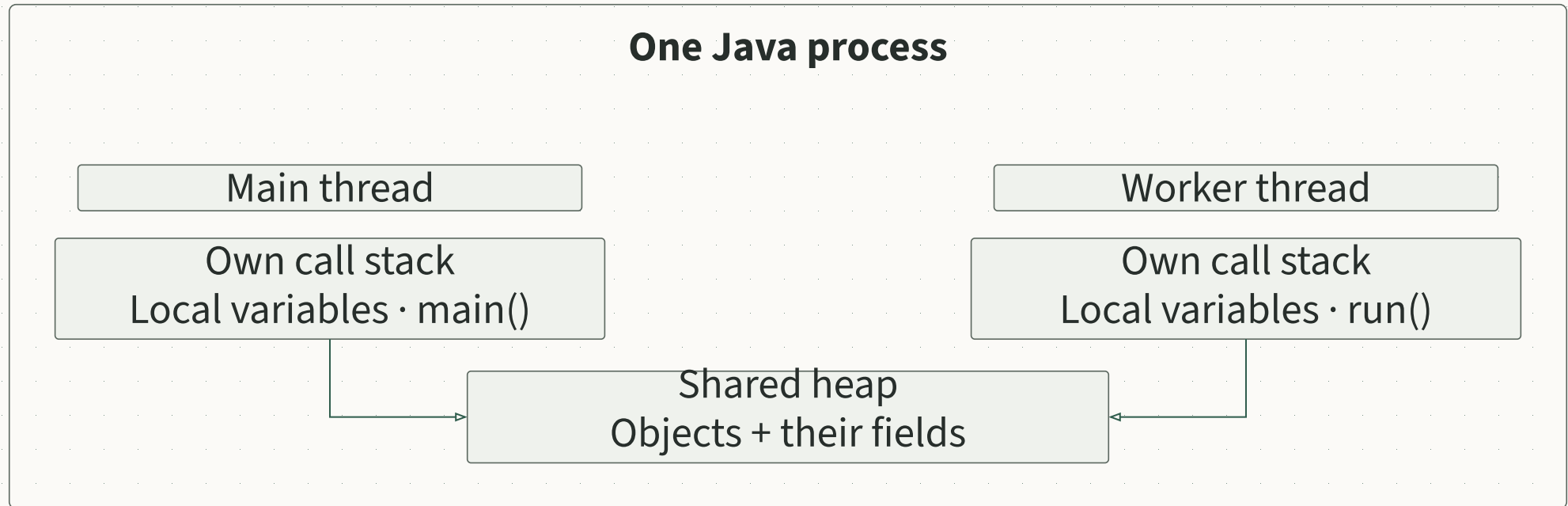
Resource conflict: one burner



Communication: hand over ingredients

More threads do not automatically make a program faster.

Separate Stacks, Shared Objects



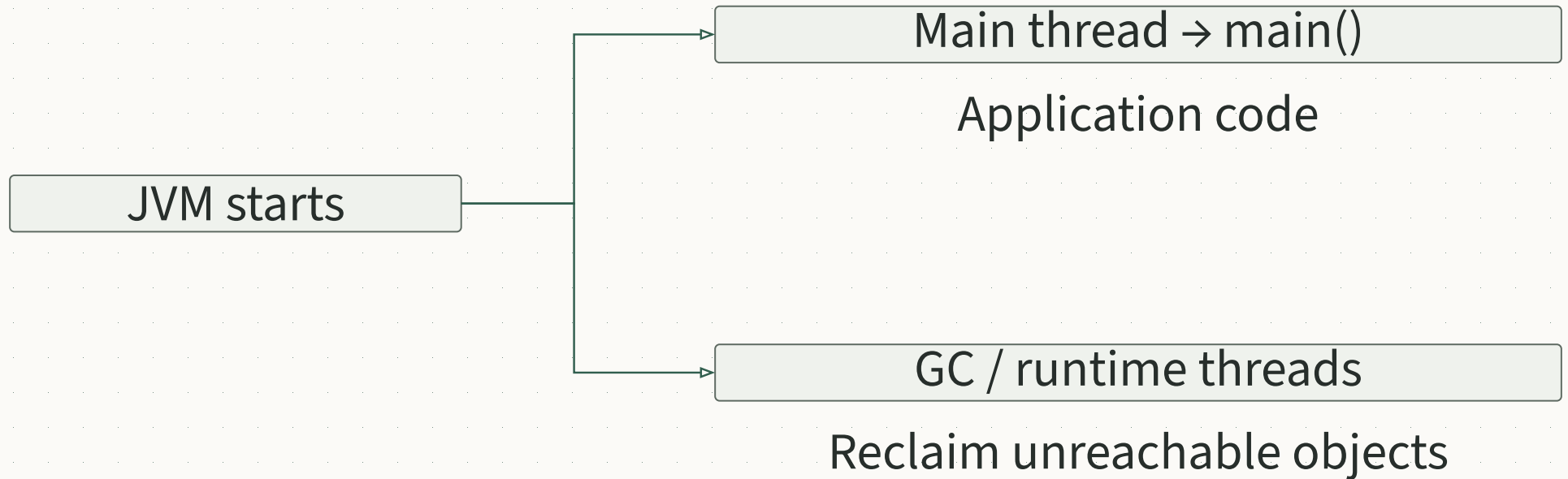
A thread is an execution path within a process. Shared objects allow communication—and conflicting access.

Which Threads Already Exist?



Besides main, which background work does the JVM perform?

Which Threads Already Exist?



Normal JVM shutdown: no live non-daemon threads remain

A worker can outlive main(). Daemon threads do not keep the JVM alive.

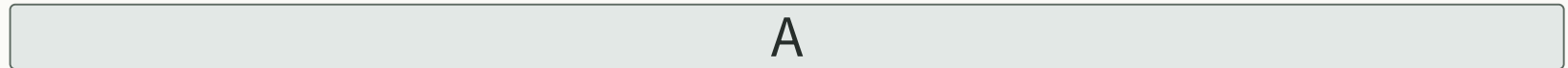
Concurrency Is Not Always Parallelism

One core



Time slicing: concurrent, not simultaneous

Core 1



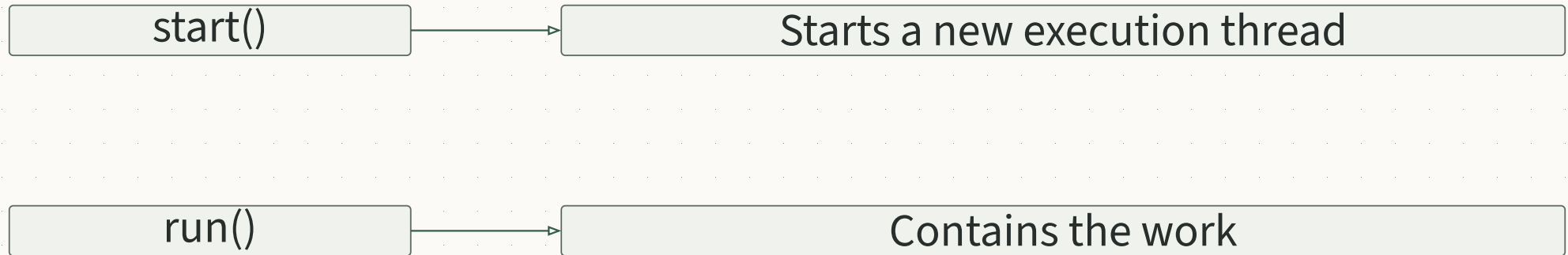
Core 2



Time → • on two cores, A and B can run in parallel

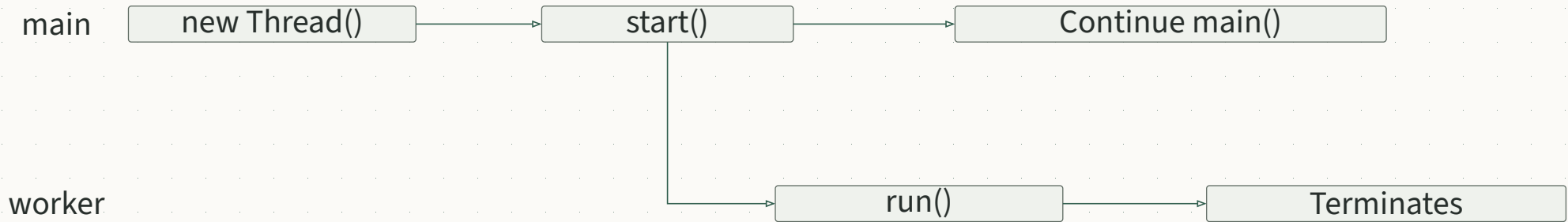
3. Class-Based Threads

start() Creates Concurrency; run() Contains Work



Calling run() directly is an ordinary call on the current thread

Object Creation Is Not Thread Start



Object creation $\neq$ thread start · a thread can be started only once

Start an Empty Thread



Task 1

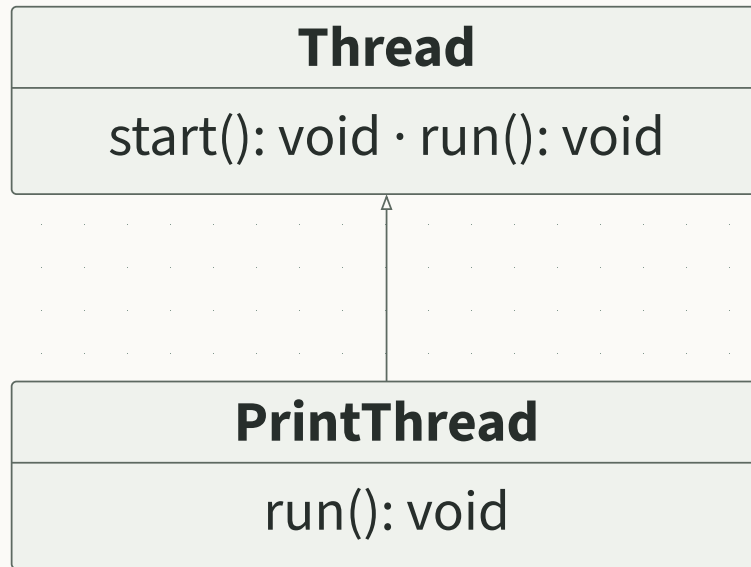
Create and start an additional thread.

```
1 public class RunThread1 {  
2     public static void main(String[] args) {  
3         Thread thread = new Thread();  
4         System.out.println("Object created");  
5         thread.start();  
6         System.out.println("Thread started");  
7     }  
8 }
```

Java

Why does the worker print nothing? No task was supplied to run().

Extend Thread and Override run()



extends Thread

Override run()
Inherit start()



Task 2

Make the worker print a message. Call `start()`, not `run()`.

A Thread Subclass

```
1 public class PrintThread extends Thread {
2     @Override
3     public void run() {
4         System.out.println("Hooray, I'm running in parallel!");
5     }
6 }
```

Java

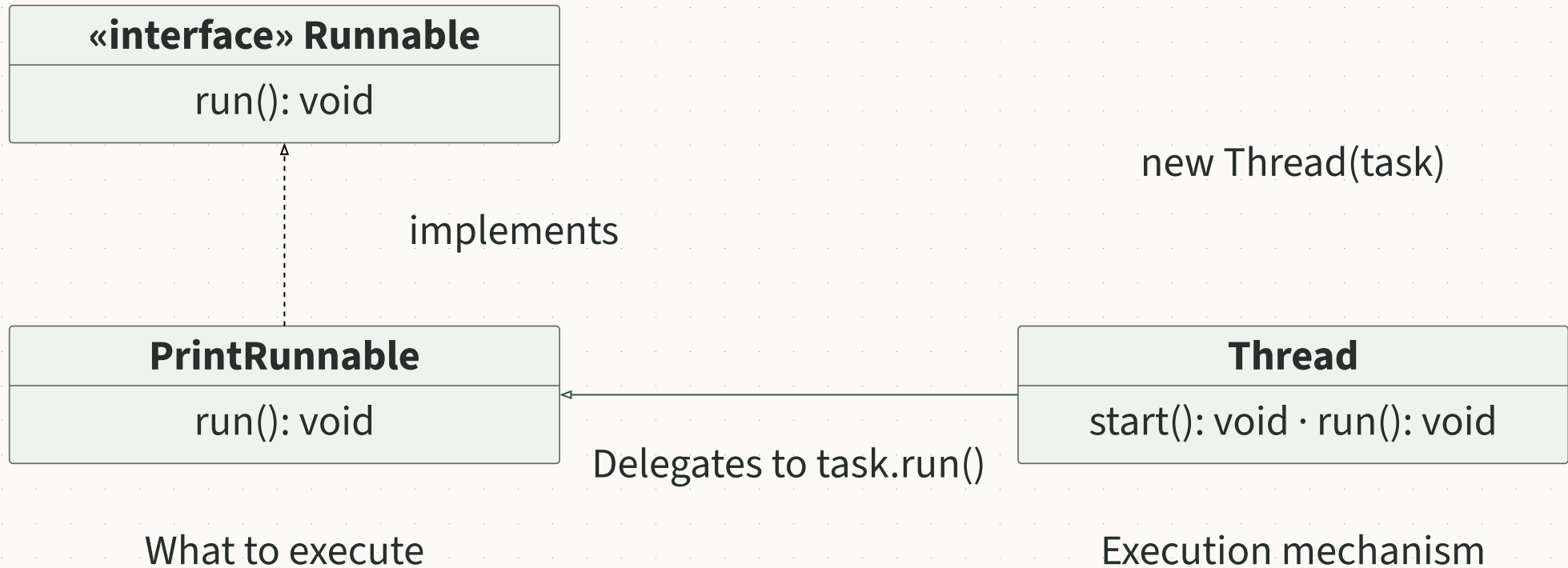
```
1 public class RunThread2 {
2     public static void main(String[] args) {
3         PrintThread thread = new PrintThread();
4         thread.start();
5         System.out.println("Thread started");
6     }
7 }
```

Java

Separate files: PrintThread.java and RunThread2.java. Which message appears first?

4. Interface-Based Threads

Separate the Task from Its Execution



Implementing `Runnable` leaves your class free to extend another class.

Implement Runnable

```
1 public class PrintRunnable implements Runnable {
2     @Override
3     public void run() {
4         System.out.println("Hooray, I'm running in parallel!");
5     }
6 }
```

Java

```
1 public class InterfaceBased {
2     public static void main(String[] args) {
3         PrintRunnable task = new PrintRunnable();
4         Thread thread = new Thread(task);
5         thread.start();
6         System.out.println("Thread started");
7     }
8 }
```

Java

Two Counters: The Worker

```
1  class CounterRunnable implements Runnable {
2      private int counter;
3      @Override
4      public void run() {
5          while (counter < 10) {
6              System.out.println("Thread counter: " + counter++);
7          }
8          System.out.println("Exiting run()");
9      }
10 }
```

Java

This counter belongs to the Runnable object.

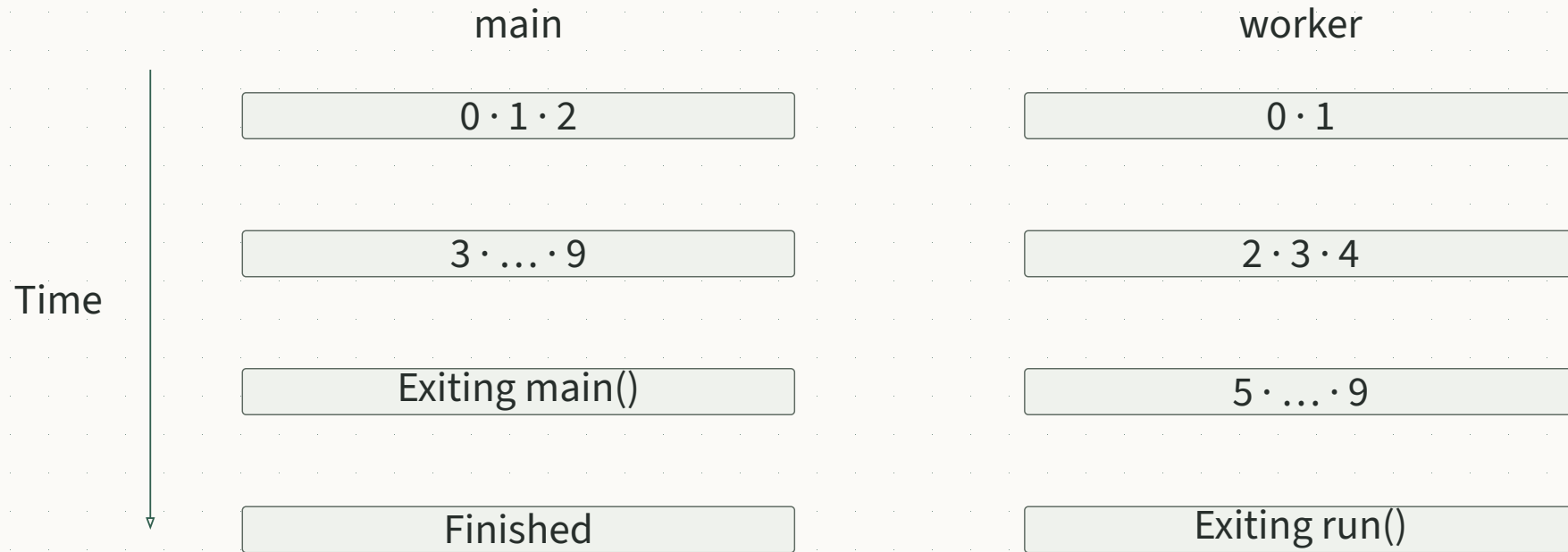
Two Counters: The Main Thread

```
1  public class Counters {
2      private static int counter;
3      public static void main(String[] args) {
4          new Thread(new CounterRunnable()).start();
5          while (counter < 10) {
6              System.out.println("Main counter: " + counter++);
7          }
8          System.out.println("Exiting main()");
9      }
10 }
```

Java

Predict the output. These are two separate counter fields, not shared state.

One Possible Interleaving



One possible interleaving · each counter still runs from 0 to 9

Neither thread is guaranteed to finish first. The worker can continue after main returns.

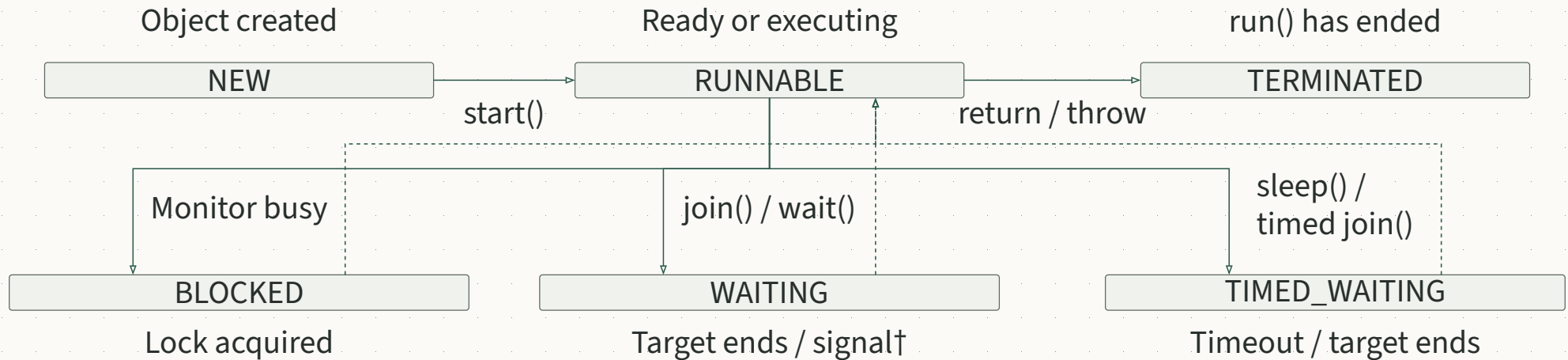
5. Thread States and Waiting

Think Like a Thread



Which states and transitions do you need when the CPU is busy, a lock is held, or another thread has not finished?

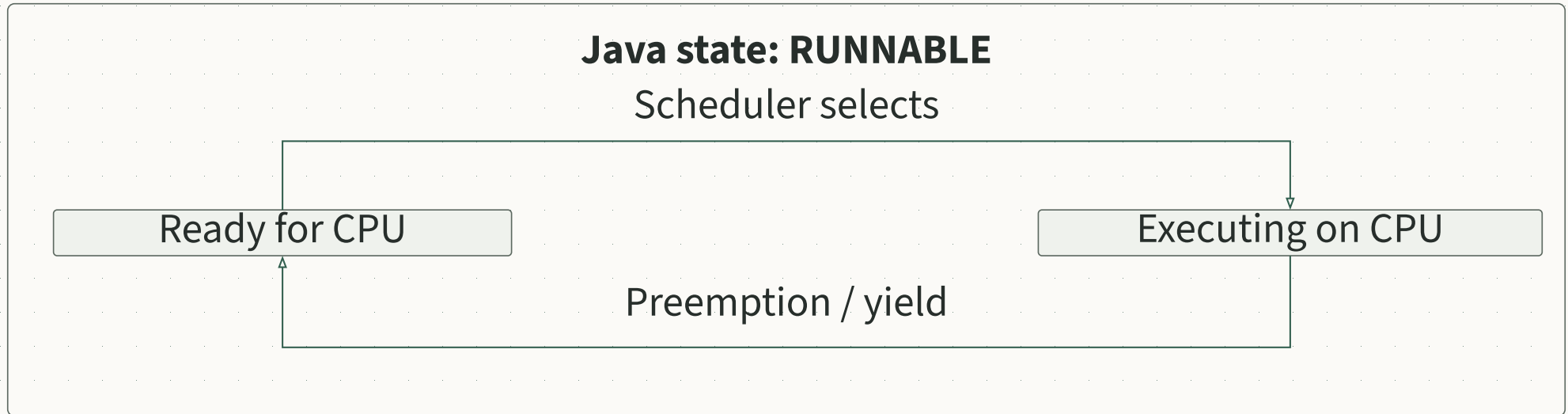
Java Thread.State



†wait() must reacquire the monitor; interruption also ends sleep / join / wait

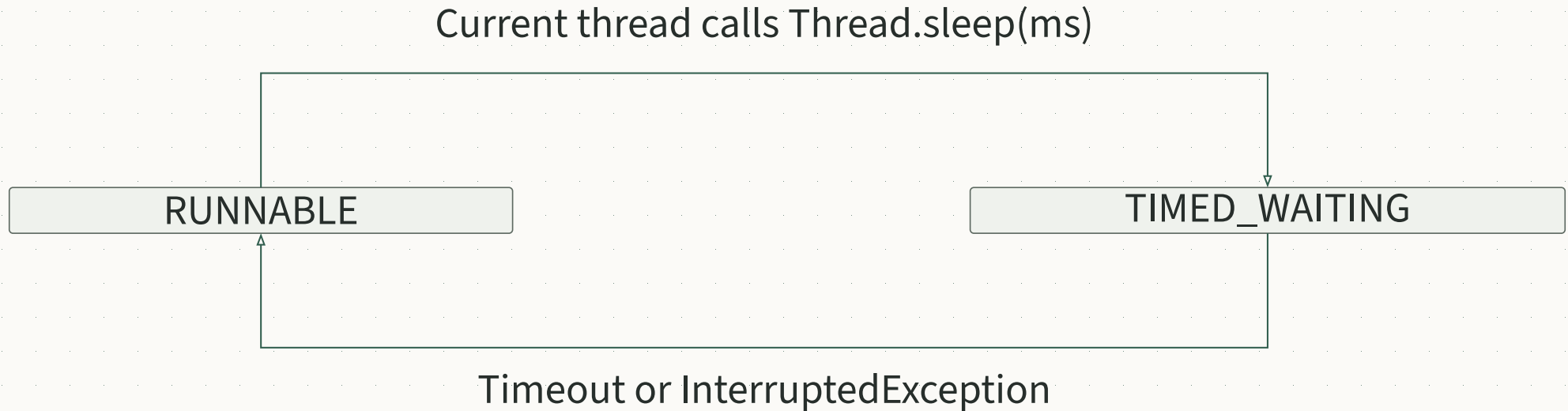
A terminated thread cannot restart; its Java object may still exist

Ready and Running Are Both RUNNABLE



No guaranteed order or fairness · `setPriority()` is only a hint

sleep() Pauses the Calling Thread



Ready again $\neq$ immediately running · `sleep()` does not release monitors

Milliseconds are a long value. Timing depends on the scheduler; it is not an exact deadline.

Handle Interruption

```
1  public static void main(String[] args) {
2      Thread worker = new Thread(new PrintRunnable());
3      worker.start();
4      try {
5          Thread.sleep(1000);
6      } catch (InterruptedException e) {
7          Thread.currentThread().interrupt();
8          return;
9      }
10     System.out.println("Main is ready again");
11 }
```

Java

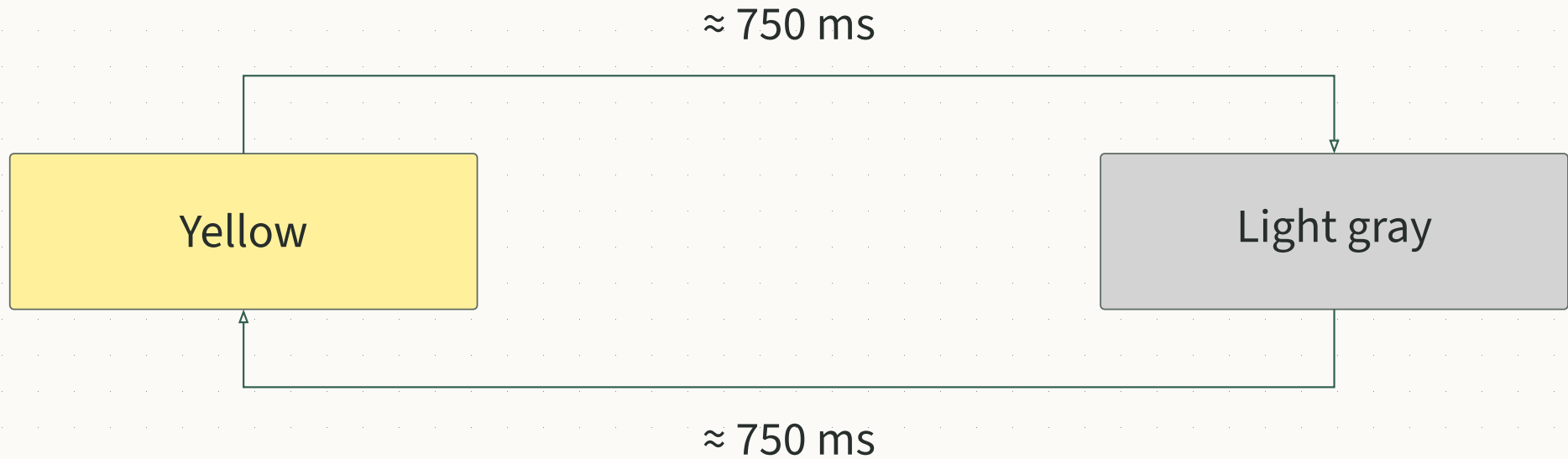
Main sleeps, not worker. Restore the interrupt flag when you cannot propagate the exception.

Task: Blink a Window

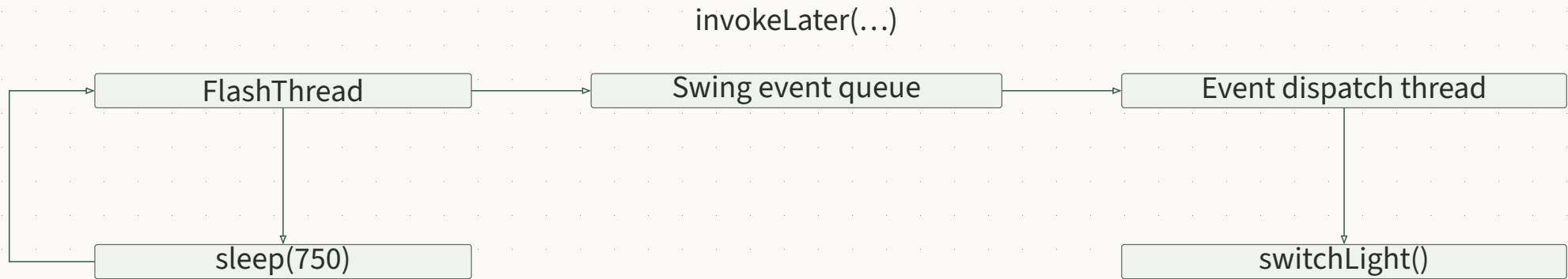


Task 3

Alternate the background between yellow and light gray about every 750 ms.



Keep Swing Updates on the Event Dispatch Thread



Worker waits · only the EDT changes Swing components

Never sleep on the EDT. A Swing Timer is simpler for real GUI animation; this exercise demonstrates a worker thread.

Build the Window on the EDT

```
1  public class FlashLight {
2      private boolean isLightOn;
3      private final JFrame frame = new JFrame("Flashing light");
4      public FlashLight() {
5          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6          frame.setSize(300, 250);
7          frame.getContentPane().setBackground(Color.LIGHT_GRAY);
8          frame.setVisible(true);
9      }
10     public void switchLight() {
11         isLightOn = !isLightOn;
12         frame.getContentPane().setBackground(
13             isLightOn ? Color.YELLOW : Color.LIGHT_GRAY);
14     }
15 }
```

Java

A Worker Schedules the Color Change

```
1  class FlashThread extends Thread {
2      private final FlashLight light;
3      FlashThread(FlashLight light) { this.light = light; }
4      public void run() {
5          while (!isInterrupted()) {
6              SwingUtilities.invokeLater(light::switchLight);
7              try {
8                  Thread.sleep(750);
9              } catch (InterruptedException e) {
10                 Thread.currentThread().interrupt();
11                 return;
12             }
13         }
14     }
15 }
```

Java

Start the Window and Worker

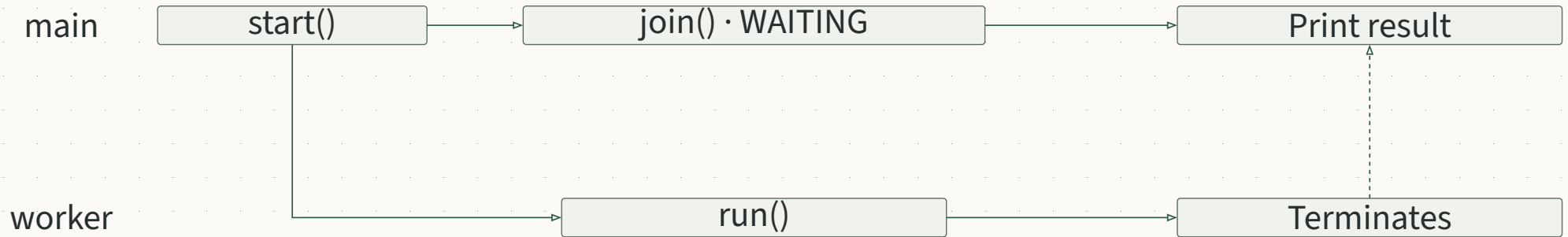
Add this method to FlashLight:

```
1 public static void main(String[] args) {  
2     SwingUtilities.invokeLater(() → {  
3         FlashLight light = new FlashLight();  
4         new FlashThread(light).start();  
5     });  
6 }
```

Java

The lambda runs on the EDT. Closing this demo window exits the application.

join() Waits for Another Thread to Finish



`join()` waits for this worker · interruption throws `InterruptedException`

Continue Only After the Worker Ends

```
1 public static void main(String[] args)
2     throws InterruptedException {
3     Thread worker = new Thread(new PrintRunnable());
4     worker.start();
5     worker.join();
6     System.out.println("We have joined!");
7 }
```

Java

`join(ms)` limits the wait; check `isAlive()` afterwards. A timeout does not stop the worker.

A Sleepy Worker

```
1  public class SleepyThread extends Thread {
2      @Override
3      public void run() {
4          for (int i = 0; i < 5; i++) {
5              System.out.println("I'm sooo tired ...");
6              try {
7                  Thread.sleep(1000);
8              } catch (InterruptedException e) {
9                  Thread.currentThread().interrupt();
10                 return;
11             }
12         }
13         System.out.println("Okay, I'm awake again.");
14     }
15 }
```

Java

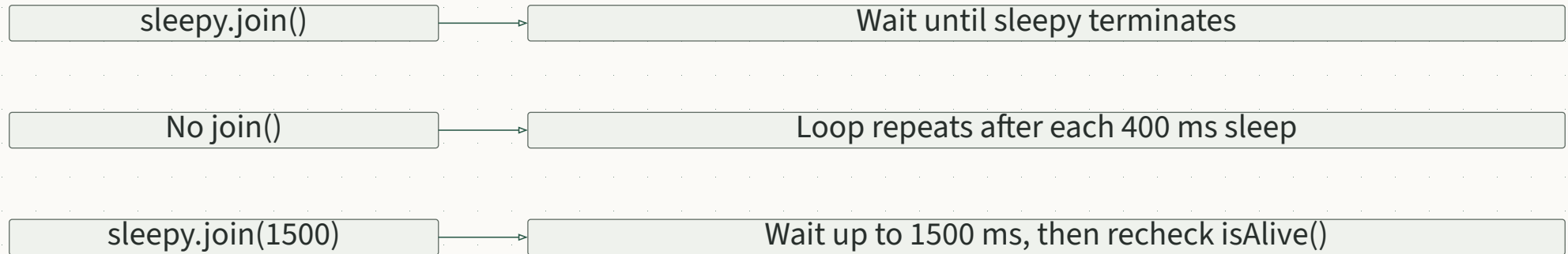
Predict the Output with and Without join()

```
1  public class JoinThreads {
2      public static void main(String[] args)
3          throws InterruptedException {
4          SleepyThread sleepy = new SleepyThread();
5          sleepy.start();
6          while (sleepy.isAlive()) {
7              System.out.println("Wake up!");
8              Thread.sleep(400);
9              sleepy.join();
10         }
11         System.out.println("At last ...");
12     }
13 }
```

Java

Compare join(), no join(), and join(1500). Which thread waits?

Three Waiting Behaviors



A timed join may return while the worker is still alive

The combined sleep and timed join make the interval roughly 1900 ms while sleepy is still alive, not exactly 1500 ms.

6. Synchronization

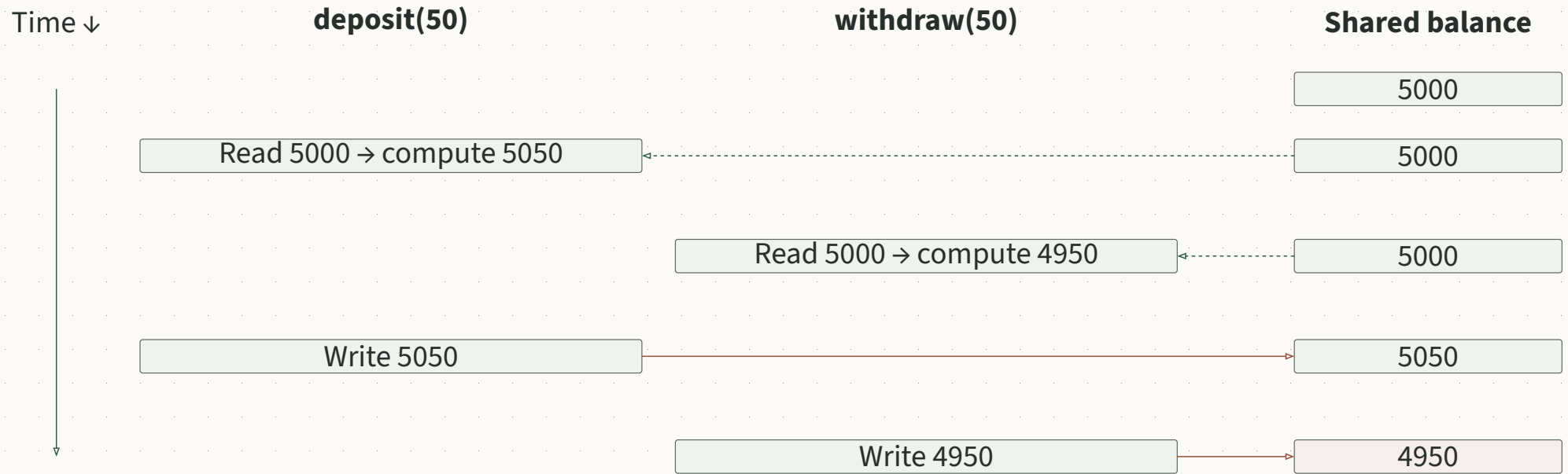
Shared Account, Concurrent Updates

```
1  public class Account {  
2      private double balance;  
3      public void deposit(double amount) {  
4          double newBalance = balance + amount;  
5          if (newBalance > balance)  
6              balance = newBalance;  
7      }  
8      public void withdraw(double amount) {  
9          double newBalance = balance - amount;  
10         if (newBalance ≥ 0.0)  
11             balance = newBalance;  
12     }  
13 }
```

Java

Starting at €5000, two threads deposit and withdraw €50. Could the balance end at €4950?
Teaching model only: real money needs decimal or integer units and input validation.

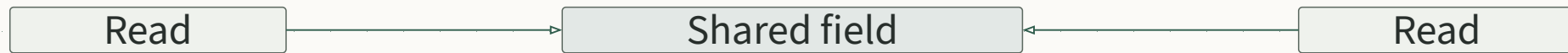
A Lost Update



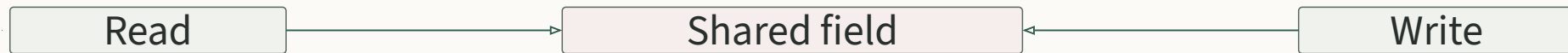
Expected: 5000 · actual: 4950 · the deposit is lost

Conflicting Access to Shared State

Two threads · same field · no synchronization



No conflicting writes



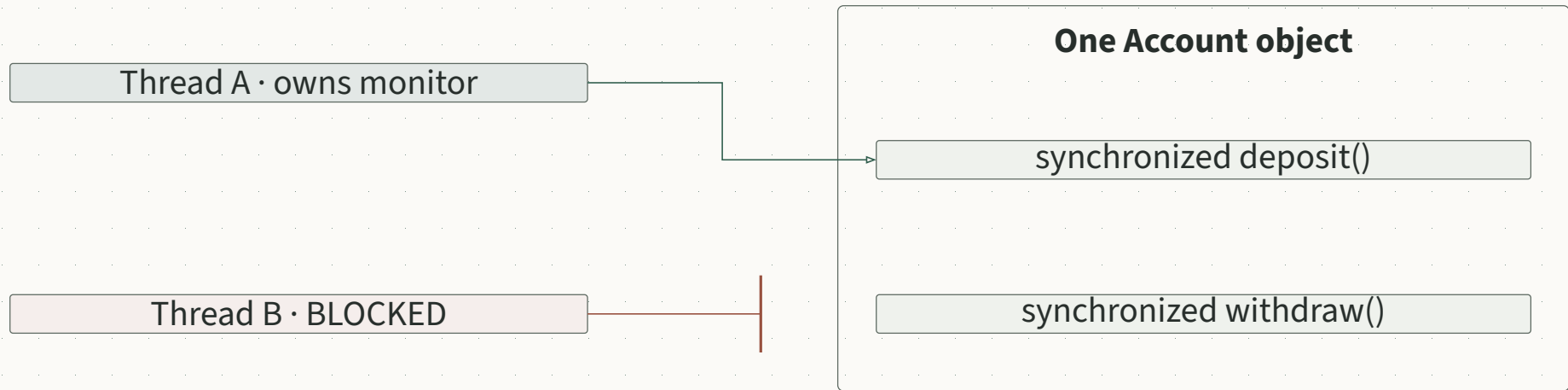
Value / visibility may depend on order



Updates can overwrite each other

A race condition makes correctness depend on timing. Unsynchronized conflicting accesses to a shared field form a data race.

synchronized Uses the Object's Monitor



Same monitor for both methods · released on return or exception



Task 4

Protect both account methods with the same monitor.

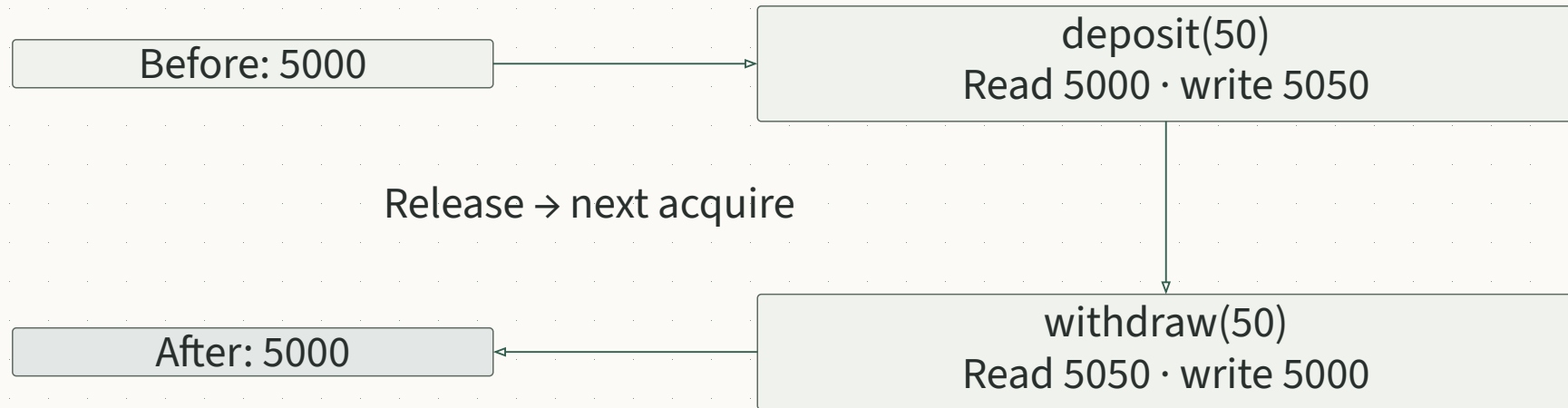
Protect the Whole Read–Check–Write

```
1  public class Account {  
2      private double balance;  
3      public synchronized void deposit(double amount) {  
4          double newBalance = balance + amount;  
5          if (newBalance > balance)  
6              balance = newBalance;  
7      }  
8      public synchronized void withdraw(double amount) {  
9          double newBalance = balance - amount;  
10         if (newBalance ≥ 0.0)  
11             balance = newBalance;  
12     }  
13 }
```

Java

Unlocking makes writes visible to the next thread that acquires the same monitor.

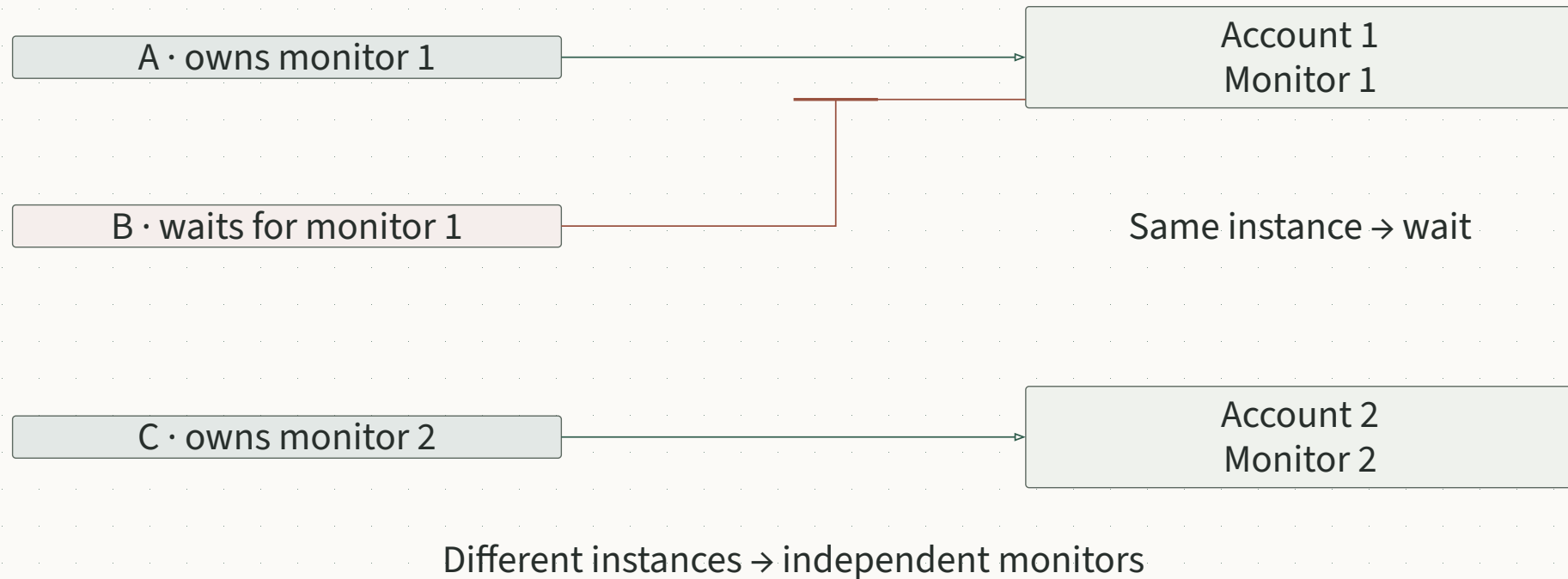
Serialize Updates, Not the Whole Program



One monitor · each read-check-write is protected

The opposite order also preserves the balance

The Monitor Belongs to the Object



Unsynchronized methods do not acquire the monitor and can still run. Protect every access to shared mutable state.

7. License Notice

Attribution

- Shared under CC BY-NC-SA 4.0.
- Based on teaching material by Prof. Dr. Marc Hensel, supplied privately.
- Text, diagrams and layout adapted; the private source cannot be linked.